

Phase 8 – Persistent Storage in EKS Using EBS CSI Driver

Objective

In this phase we will learn how Kubernetes stores application data permanently using AWS EBS volumes.

By default, Kubernetes pods are ephemeral.

This means:

Pod Deleted

↓

Data Deleted

If an application stores data inside the container filesystem and the pod gets recreated, all data is lost.

To solve this problem, Kubernetes provides:

Persistent Volumes (PV)

Persistent Volume Claims (PVC)

Storage Classes (SC)

These components allow applications to store data outside the pod lifecycle.

In AWS EKS, persistent storage is typically backed by:

Amazon EBS Volumes

managed through:

AWS EBS CSI Driver

What We Will Build

Application Pod

▼

PersistentVolumeClaim (PVC)

▼

PersistentVolume (PV)

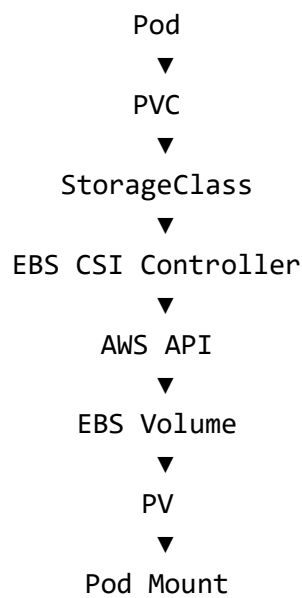
▼

AWS EBS Volume

At the end of this phase we will:

- Install EBS CSI Driver
- Configure IAM permissions
- Create StorageClass
- Create PVC
- Dynamically provision EBS volumes
- Mount storage into pods
- Validate persistent storage
- Understand reclaim policies

Architecture



Why Do We Need StorageClass, PVC and PV?

Many beginners get confused here.

Think of it like ordering a hard disk.

StorageClass

StorageClass defines: What type of storage should be created?

Example:

gp3
gp2
io1
io2

StorageClass acts like a template.

Example:

Create me GP3 storage

Use EBS CSI Driver

Delete storage when claim is removed

[PVC \(Persistent Volume Claim\)](#)

PVC is a request.

Example:

I need 4 GB storage.

Application teams create PVCs.

They do not create EBS volumes directly.

[PV \(Persistent Volume\)](#)

PV is the actual storage resource created by Kubernetes.

Example:

4 GB EBS Volume

PV is automatically created by the CSI Driver.

[Step 1 – Verify Existing Storage Classes](#)

Check available storage classes.

```
kubectl get storageclass
```

Output:

NAME	PROVISIONER
gp2	kubernetes.io/aws-ebs

This is the legacy AWS storage provisioner.

Modern EKS uses:

`ebs.csi.aws.com`

through the EBS CSI Driver.

Step 2 – Verify EBS CSI Driver Addon

Check installed EKS addons.

```
aws eks list-addons `
  --cluster-name mani-eks-cluster
```

Output before installation:

```
{
  "addons": [
    "coredns",
    "kube-proxy",
    "vpc-cni"
  ]
}
```

Notice:

```
aws-ebs-csi-driver
is missing.
```

Step 3 – Verify OIDC Provider

The CSI Driver needs AWS permissions.

Verify OIDC:

```
aws eks describe-cluster `
  --name mani-eks-cluster `
  --query "cluster.identity.oidc.issuer" `
  --output text
```

Output:

<https://oidc.eks.ap-south-1.amazonaws.com/id/3CC36F32A671BC8BB549EB65D28F4998>

OIDC allows Kubernetes service accounts to assume AWS IAM roles.

Step 4 – Create IAM Trust Policy

Create:

```
ebs-csi-trust-policy.json
```

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::<ACCOUNT-ID>:oidc-provider/<OIDC-PROVIDER>"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "<OIDC-PROVIDER>:aud": "sts.amazonaws.com",
          "<OIDC-PROVIDER>:sub": "system:serviceaccount:kube-
system:ebs-csi-controller-sa"
        }
      }
    }
  ]
}
```

Step 5 – Create IAM Role

```
aws iam create-role `
  --role-name AmazonEKS_EBS_CSI_DriverRole `
  --assume-role-policy-document file:///ebs-csi-trust-policy.json
```

Step 6 – Attach AWS Managed Policy

```
aws iam attach-role-policy `
  --role-name AmazonEKS_EBS_CSI_DriverRole `
  --policy-arn arn:aws:iam::aws:policy/service-
role/AmazonEBSCSIDriverPolicy
```

This policy allows the CSI Driver to:

- Create EBS Volumes
- Delete EBS Volumes
- Attach Volumes
- Detach Volumes
- Describe Volumes

Step 7 – Install EBS CSI Driver Addon

```
aws eks create-addon `
  --cluster-name mani-eks-cluster `
  --addon-name aws-ebs-csi-driver `
  --service-account-role-arn arn:aws:iam::<ACCOUNT-
ID>:role/AmazonEKS_EBS_CSI_DriverRole
```

Verify:

```
aws eks list-addons `
  --cluster-name mani-eks-cluster
```

Output:

```
aws-ebs-csi-driver
coredns
kube-proxy
vpc-cni
```

Step 8 – Verify CSI Components

```
kubectl get pods -n kube-system
```

Output:

```
ebs-csi-controller-xxxxx
ebs-csi-controller-xxxxx
```

```
ebs-csi-node-xxxxx
ebs-csi-node-xxxxx
```

CSI Components Explained

ebs-csi-controller

Runs as Deployment.

Responsibilities:

- Watch PVCs
- Create EBS Volumes
- Create PVs
- Delete EBS Volumes
- Manage Storage Lifecycle

Think of it as: Storage Brain

`ebs-csi-node`

Runs as DaemonSet.

One pod per worker node.

Responsibilities:

Attach EBS Volume

Mount EBS Volume

Expose Volume To Pods

Think of it as: Storage Hands

Step 9 – Create StorageClass

Create: `storageclass.yaml`

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
  name: ebs-sc
```

```
provisioner: ebs.csi.aws.com
```

```
volumeBindingMode: WaitForFirstConsumer
```

```
parameters:
```

```
  type: gp3
```

Apply: `kubectl apply -f storageclass.yaml`

Verify: `kubectl get storageclass`

What Is WaitForFirstConsumer?

Without this:

PVC Created

↓

EBS Created Immediately

With:

PVC Created



Wait



Pod Created



Scheduler Chooses Node



Create EBS In Correct AZ

This prevents Availability Zone mismatches.

Step 10 – Create PVC

Create: pvc.yaml

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: ebs-pvc

spec:

accessModes:

- ReadWriteOnce

storageClassName: ebs-sc

resources:

requests:

storage: 4Gi

Apply: `kubectl apply -f pvc.yaml`

Verify: `kubectl get pvc`

Output: Pending

Expected behavior.

No pod is consuming the claim yet.

Step 11 – Create Pod Using PVC

Create: ebs-pod.yaml

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: ebs-demo
```

```
spec:
```

```
  containers:
```

```
    - name: app
```

```
      image: nginx
```

```
  volumeMounts:
```

```
    - name: ebs-storage
```

```
      mountPath: /data
```

```
  volumes:
```

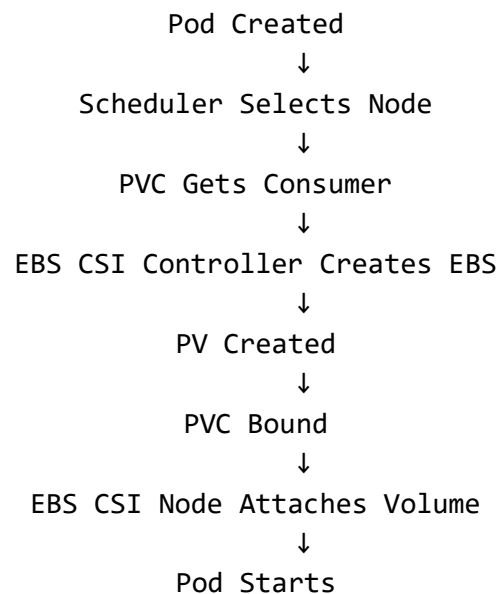
```
    - name: ebs-storage
```

```
      persistentVolumeClaim:
```

```
        claimName: ebs-pvc
```

Apply: `kubectl apply -f ebs-pod.yaml`

What Happens Behind The Scenes?



Step 12 – Verify Dynamic Provisioning

Verify PVC: `kubectl get pvc`

Output: Bound

Verify PV: `kubectl get pv`

Output: pvc-xxxxxxx

Verify AWS Volume:

```
aws ec2 describe-volumes `
  --filters "Name=status,Values=available,in-use" `
  --query "Volumes[*].[VolumeId,Size,State]" `
  --output table
```

Observe newly created:

4 GB EBS Volume

Step 13 – Persistence Test

Create file: `kubectl exec -it ebs-demo -- sh`

Inside pod:

```
echo "Hello from EBS Persistent Volume" > /data/app.txt
```

```
cat /data/app.txt
```

```
exit
```

Step 14 – Delete Pod

```
kubectl delete pod ebs-demo
```

Verify: `kubectl get pvc`

```
kubectl get pv
```

Storage still exists.

Step 15 – Recreate Pod

```
kubectl apply -f ebs-pod.yaml
```

Verify: `kubectl exec -it ebs-demo -- cat /data/app.txt`

Output: Hello from EBS Persistent Volume

Success.

Data survived pod deletion.

Access Modes

ReadWriteOnce (RWO)

One Node

Read + Write

Used by EBS.

ReadOnlyMany (ROX)

Many Nodes

Read Only

ReadWriteMany (RWX)

Many Nodes

Read + Write

Typically provided by EFS.

Reclaim Policy

Current StorageClass:

RECLAIMPOLICY = Delete

Behavior:

Delete PVC

↓

Delete PV

↓

Delete EBS Volume

You demonstrated this practically.

Commands executed:

```
kubect1 delete pod ebs-demo
```

```
kubect1 delete pvc ebs-pvc
```

Result: `kubectl get pv`

Output: No resources found

The EBS volume was deleted automatically.

Retain Policy

Example: `reclaimPolicy: Retain`

Behavior:

Delete PVC



PV Released



EBS Volume Preserved



Data Preserved

Commonly used for:

MySQL

PostgreSQL

MongoDB

Production Databases

Important Note About Reclaim Policy

Reclaim Policy is configured in:

StorageClass

NOT:

PVC

Pod

Also note:

Once a PV is created from a StorageClass, changing the StorageClass later does not automatically modify already-created volumes.

New PVs will use the updated StorageClass settings, but existing PVs keep the policy that was applied when they were created.

Key Learning Outcomes

By completing this phase you learned:

- Why pods are ephemeral
- Why persistent storage is required
- Difference between PV, PVC and StorageClass
- How EBS CSI Driver works
- How IAM and OIDC integrate with Kubernetes
- Dynamic volume provisioning
- Data persistence across pod recreation
- Access modes (RWO, ROX, RWX)
- Reclaim policies (Delete vs Retain)
- How EKS dynamically creates AWS EBS volumes from Kubernetes objects